Protecting hospital cloud systems when the protectors themselves can be attacked

A plain-English guide to the Distributed Cyber-Resilience Platform, for a presentation to a non-expert audience

What to remember in one minute. A clinic's patient portal, login service, health-record service, database, laboratory-results service and medication service run in the cloud. If one of them is taken over, someone must cut it off quickly, replace it with a clean copy, check the copy, and bring it back carefully. Normally one central program decides. If that program is fooled, hacked or stops, the whole clinic is exposed. This project replaces the single decision-maker with **four identical security agents** that must **agree, three out of four,** before anything is cut off. One liar or one failure cannot cause harm or stop protection.

Honest status. The agents, the signatures, the 3-of-4 agreement and the recovery pipeline are real code. They were tested many times against a **simulator** (section 6) and then run once on a **test Kubernetes cluster that the author built in a cloud sandbox** (section 7, labelled SANDBOX-CLUSTER and kept apart from the simulator numbers). **Nothing has yet been verified by the presenter on their own cluster**; the live-verified table in section 8 is empty on purpose and is for the presenter to fill with their own runs. All patient data is **synthetic**: 200 invented patients from the Synthea generator, never real people (section 5).

1. The problem, in plain English

Healthcare organisations run applications in the cloud: a portal where patients and staff look at records, a service that checks who may log in, a service that serves the health records, a database that stores them, and services for laboratory results and medications. Attackers go after exactly these. When one is compromised, three things matter:

- **Speed**: cut the compromised system off before it leaks or spreads.
- **Safety**: never cut off a healthy system by mistake, because clinicians then cannot see records.
- **Care in coming back**: replace it with a clean copy, prove the copy is clean, and give access back step by step, not all at once.

The usual design has one central controller making these decisions. That controller becomes the single weak point: if it is tricked into accusing a healthy service, it cuts it off; if it crashes, nothing is protected. In the simulator, a lying central controller cut off a healthy service **every time (20 of 20 trials)**, and with the controller down a real attack was **never** contained (0 of 20).

2. What I built

Six healthcare workloads (A to F) are watched by **four identical security agents (A to D)**. Every agent watches every workload and votes on every one; the agents are **identical peers**: no agent is the boss, and the rule stays **3 of 4**.

Workload	Name	What it does for patients and staff	If it is isolated
A	Patient Portal	shows patient records	no other service depends on it; the portal itself is unavailable
B	Login & Access Service	signs people in and checks their access	the Patient Portal and the EHR API cannot work
C	Electronic Health Records (EHR) API	serves patient records	the Patient Portal cannot show patient records
D	Clinical Database	stores patient records	the EHR API, the laboratory and medication services cannot serve records, so the portal cannot show them
E	Laboratory Results Service	serves laboratory results	the Patient Portal cannot show laboratory results
F	Medication Service	serves medication lists	the Patient Portal cannot show medications

The dashboard shows this same table, and works out the last column from which service calls which (checked against the application code), so nothing is typed in by hand. The suggested recovery order shown beside it is only a suggestion for a human operator; it does not change what the agents do, and there are no invented clinical urgency scores.

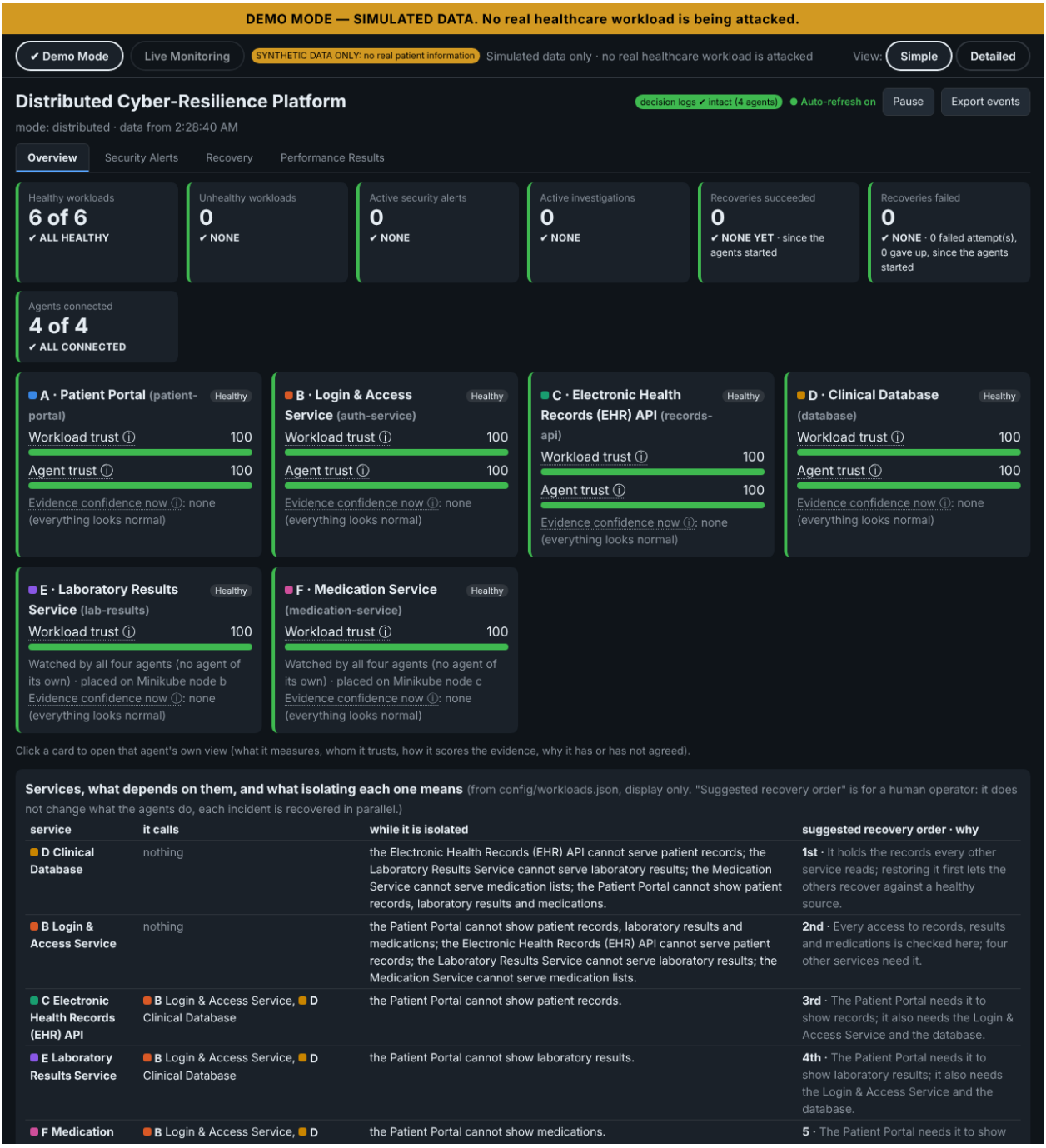
How a decision is made

1. **See.** Each agent watches its workload and its neighbours: *Network Activity* (unexpected connections), *Running Processes* (programs that should not be there), *File Changes* (files that differ from the known-good copy) and *Failed Logins*.
2. **Share evidence.** Agents send each other signed evidence. A forged or tampered message is rejected and the sender loses trust.
3. **Agree (3 of 4).** Isolating a workload needs at least three agents to sign a vote. A missing vote is never counted as approval.

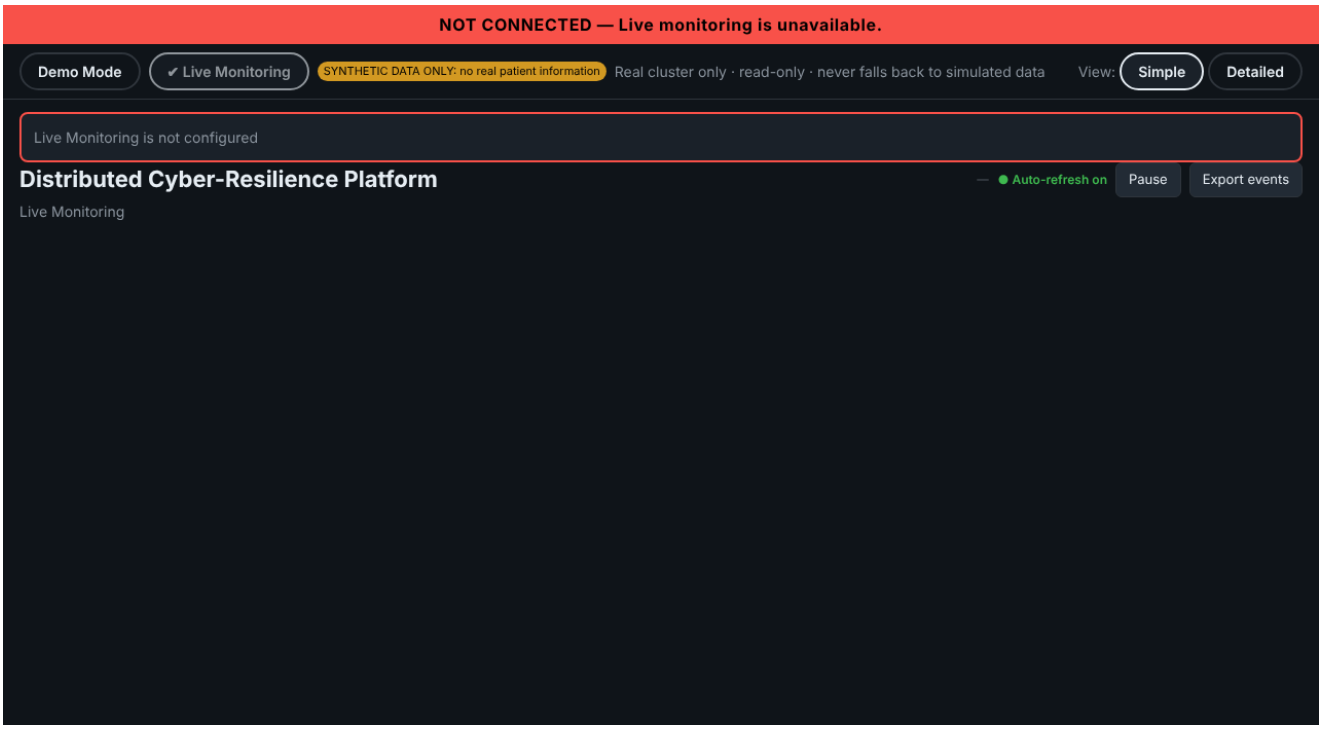
4. **Isolation.** The workload is cut off from the network (a Kubernetes network policy), and a read-only copy of the evidence is saved first.
5. **Replace and check.** A clean copy is started from the known-good image. The **Recovery Check** tests that it is healthy, that its files match, that it behaves normally and that it is really new.
6. **Return in steps.** Limited Access, Monitored Access, Confirmed by Peers, then **Normal Access Restored**, each step only when the evidence supports it. If recovery keeps failing, the workload stays isolated and a person is asked to step in.

3. Two pages that are never mixed: Demo Mode and Live Monitoring

The dashboard has a toggle at the top. **Demo Mode** shows the simulator and has the scenario buttons. **Live Monitoring** shows only the real cluster and has no buttons at all. The two pages share no data, and the server (not just the page) refuses to show simulated data on the live page. If the cluster cannot be reached the live page says so instead of showing anything else.



Demo Mode (simulator), in the new **Simple view** (the default). Amber banner: simulated data, no real healthcare workload is attacked. One top bar holds the banner, the Demo/Live toggle, the 'synthetic data only' label and the Simple/Detailed switch; below it the counters and the six workload cards.



Live Monitoring with no cluster reachable (real behaviour of the current build): it says NOT CONNECTED once, with the reason, and shows nothing from the simulator. When connected it shows a green banner with the cluster name, how many of the four agents answer and how fresh the data is.

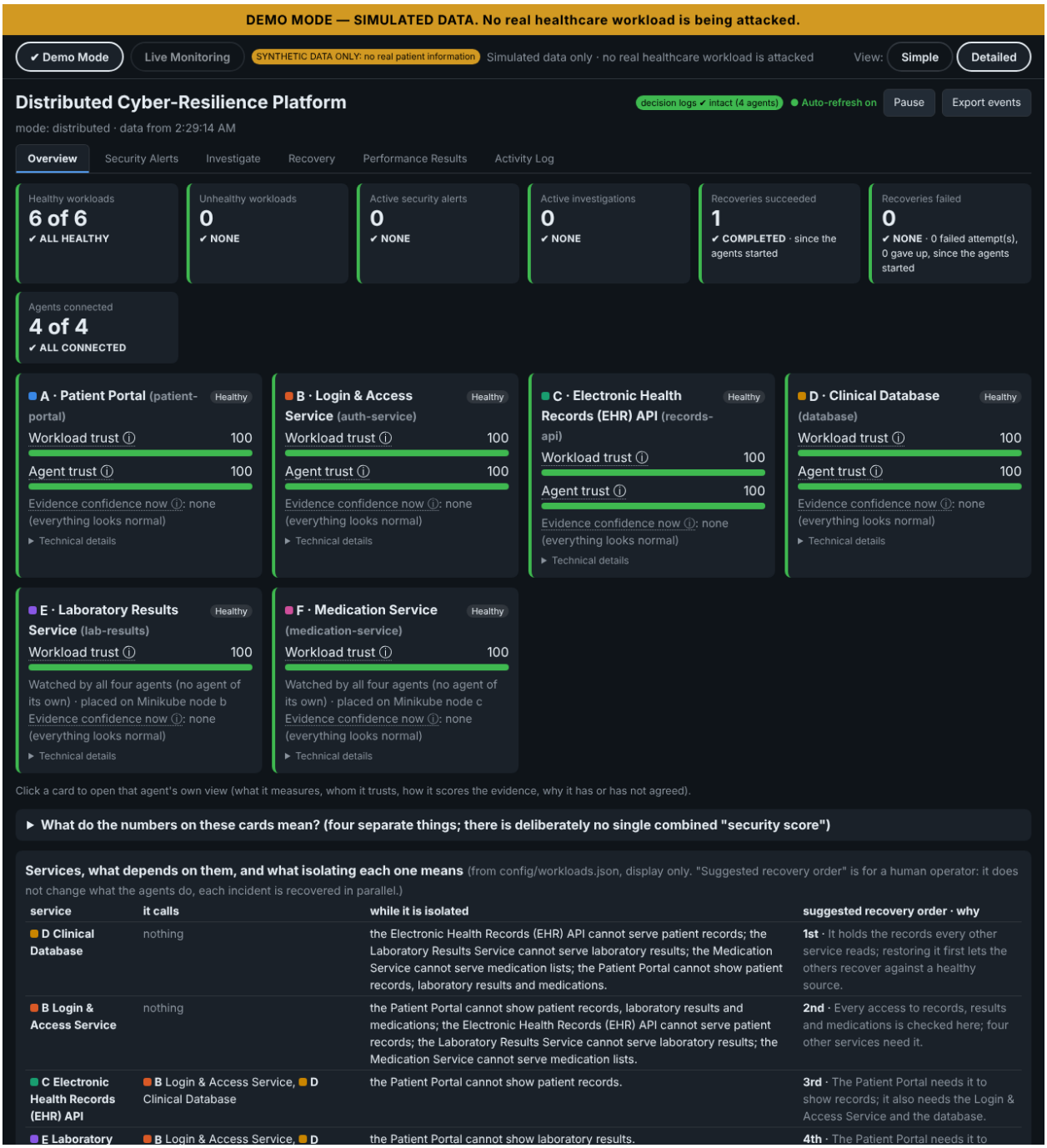
Placeholder: Live Monitoring, connected, on your own cluster. A screenshot of the connected live page from **your** run can only be taken on your cluster. Presenter: paste your own screenshot here after your run (see docs/LIVE_DEMO.md). None is included because none has been taken by the presenter; the author's sandbox-cluster screenshots are in section 7 and are labelled as such.

4. Reading the dashboard

Services, what depends on them, and what isolating each one means (from config/workloads.json, display only. "Suggested recovery order" is for a human operator: it does not change what the agents do, each incident is recovered in parallel.)

service	it calls	while it is isolated	suggested recovery order · why
D Clinical Database	nothing	the Electronic Health Records (EHR) API cannot serve patient records; the Laboratory Results Service cannot serve laboratory results; the Medication Service cannot serve medication lists; the Patient Portal cannot show patient records, laboratory results and medications.	**1st** · It holds the records every other service reads; restoring it first lets the others recover against a healthy source.
B Login & Access Service	nothing	the Patient Portal cannot show patient records, laboratory results and medications; the Electronic Health Records (EHR) API cannot serve patient records; the Laboratory Results Service cannot serve laboratory results; the Medication Service cannot serve medication lists.	**2nd** · Every access to records, results and medications is checked here; four other services need it.
C Electronic Health Records (EHR) API	**B** Login & Access Service, **D** Clinical Database	the Patient Portal cannot show patient records.	**3rd** · The Patient Portal needs it to show records; it also needs the Login & Access Service and the database.
E Laboratory Results Service	**B** Login & Access Service, **D** Clinical Database	the Patient Portal cannot show laboratory results.	**4th** · The Patient Portal needs it to show laboratory results; it also needs the Login & Access Service and the database.
F Medication Service	**B** Login & Access Service, **D** Clinical Database	the Patient Portal cannot show medications.	**5** · The Patient Portal needs it to show medications; it also needs the Login & Access Service and the database.
A Patient Portal	**B** Login & Access Service, **C** Electronic Health Records (EHR) API, **E** Laboratory Results Service, **F** Medication Service	no other service depends on it; its own job (show patient records, laboratory results and medications) stops.	**6** · Nothing else depends on it; once the services it calls are healthy it works again.

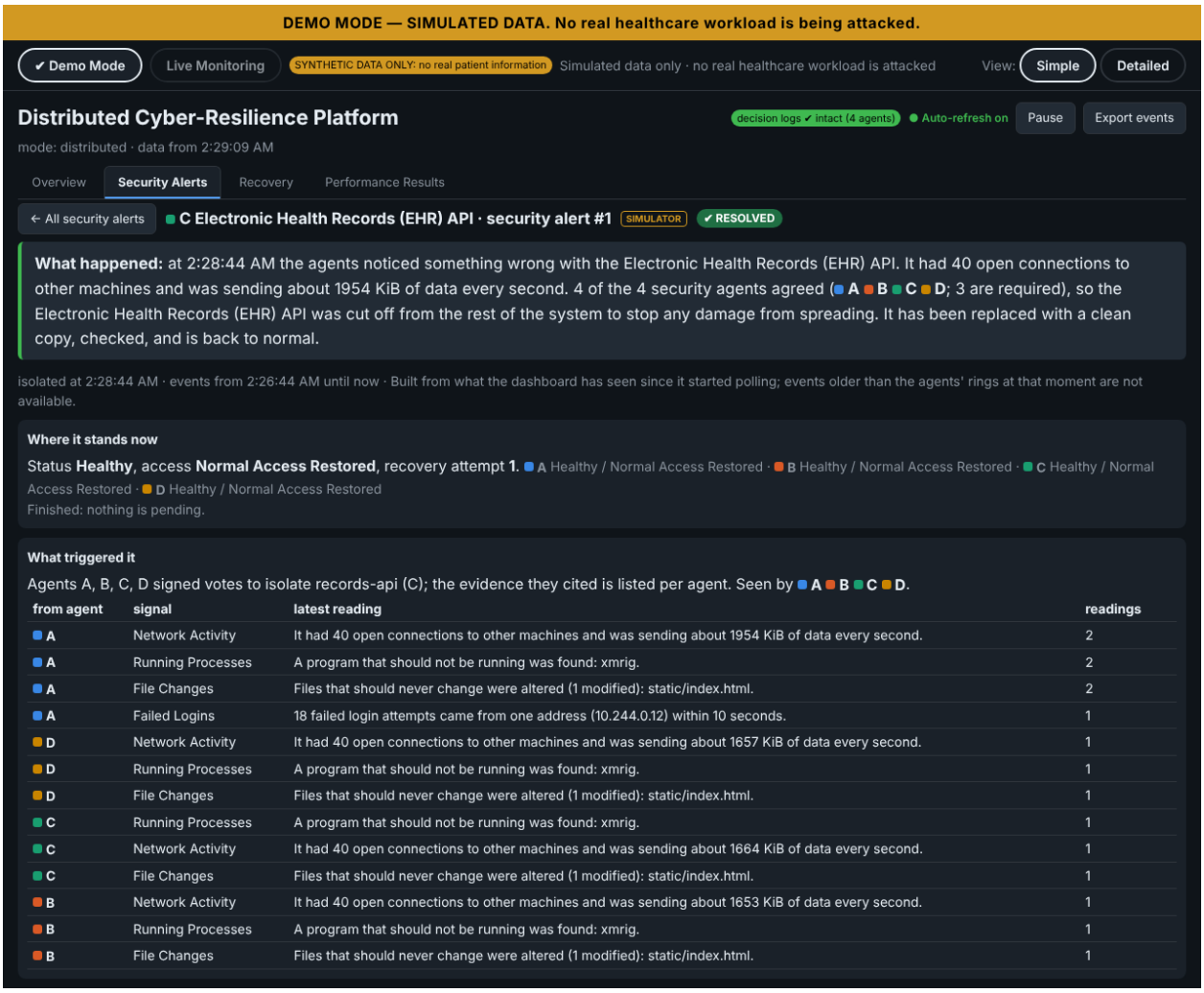
The Services table: what each of the six services calls and what stops working if it is isolated, worked out from the application code. In Simple view the technical boxes are hidden; **Detailed** shows them all.



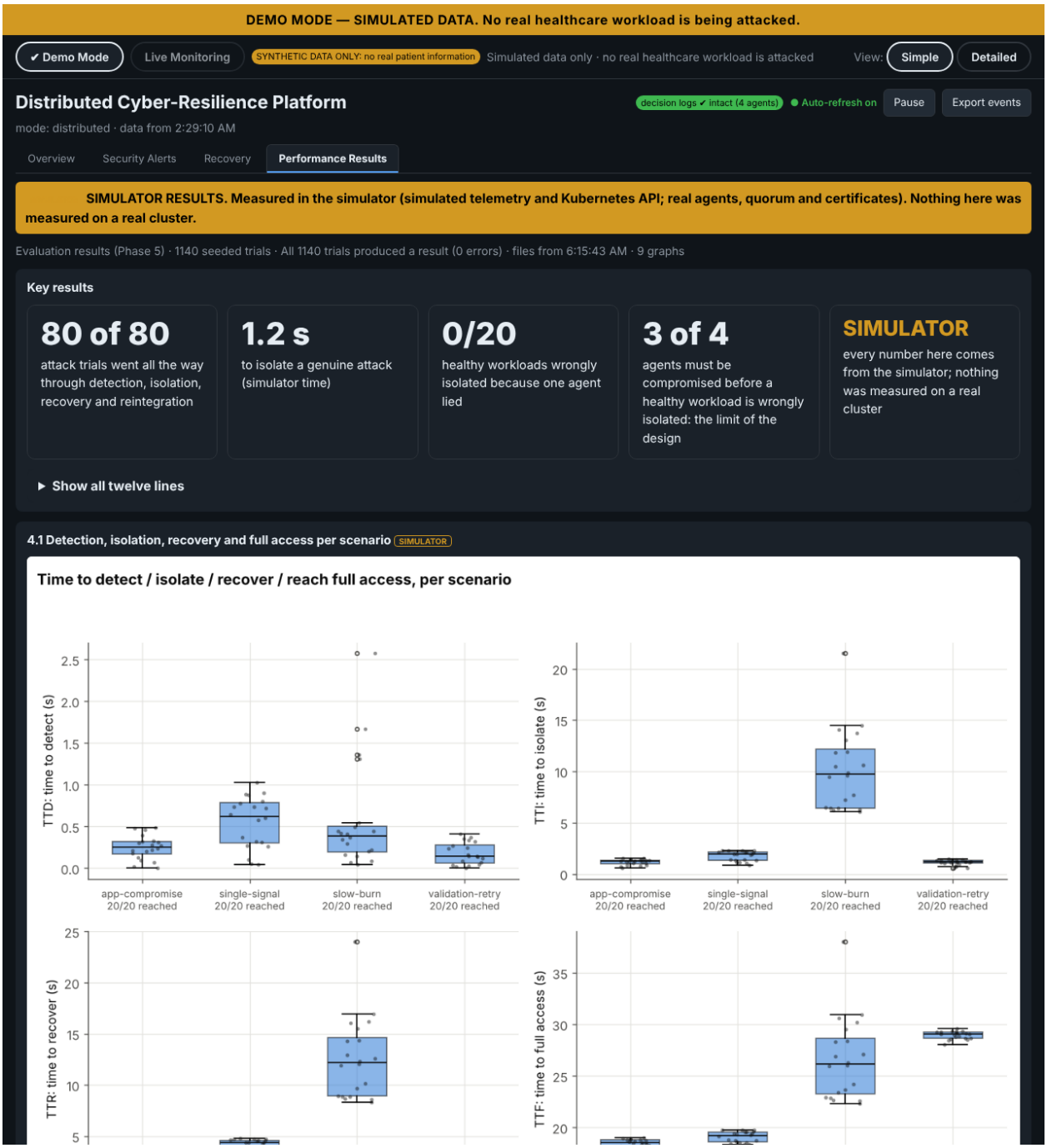
The same page in the **Detailed** view: every explanation box and technical fold is shown.

You see	It means
Workload trust (0-100)	how much the agents trust this workload right now; falls while something looks wrong, restarts at 0 on isolation, must climb back before each step of access is restored
Agent trust (0-100)	how much the other agents trust this agent's evidence; falls only if its claims are contradicted
Evidence confidence (0 to 1)	how sure one agent is about one kind of signal on one workload; not a trust score
Recovery Check	pass or fail, not a score
Agreement Reached (3 of 4)	at least three agents signed; the only way anything is isolated

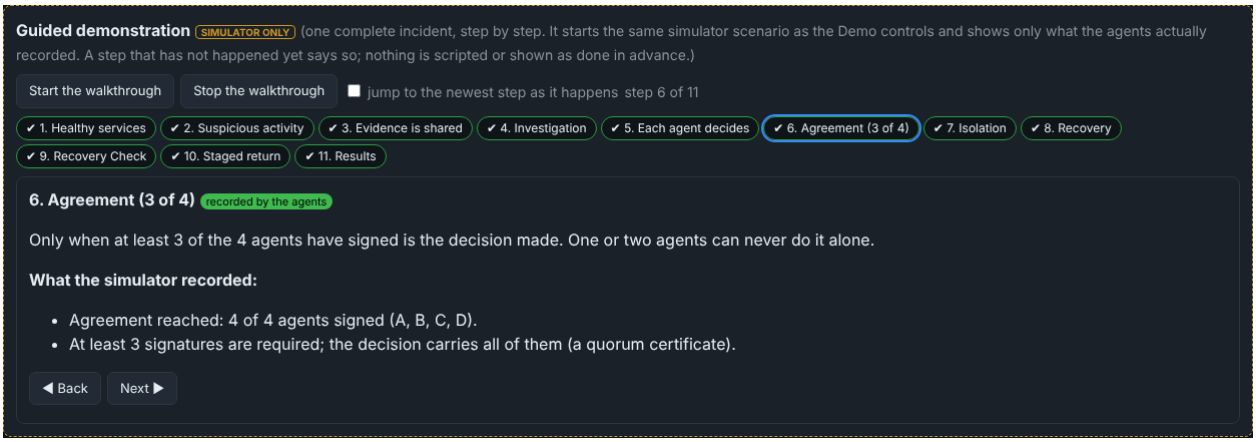
These are four separate things on purpose: there is no single combined 'security score'. Each agent is shown in exactly one of five states: **Agrees, Disagrees, Unavailable, Not yet responded, Message failed verification**. Only a counted, signed vote is 'Agrees'.



A **Security Alert** in the Simple view: it starts with a plain-English story (what happened, who agreed, how it ended) and shows each reading as a sentence; votes, trust changes and raw logs are in the Detailed view.



Performance Results: five large key-result cards (the numbers are read out of the results text); the full twelve lines, graphs and tables are one click away. Labelled as simulator results.



The **guided walkthrough** (Demo Mode only): eleven steps that run a real simulator scenario; a step that has not been reached yet says so instead of showing results.

5. Realistic data and live traffic (Synthea)

Every patient is invented. The 200 patients come from **Synthea**, the open-source synthetic patient generator (Apache-2.0, by MITRE): 491 conditions, 2,359 laboratory results and 494 medications. Social security numbers, addresses and other identifiers were left out on purpose. Every service answer says `source: Synthea, synthetic: true`. No real patient data is used anywhere.

How it flows, in real time

- The **Clinical Database** holds the dataset; the **EHR API**, **Laboratory Results** and **Medication** services read from it over the network, each after a token check with the Login service, and the **Patient Portal** calls them.
- A **client** runs three simulated clinicians who log in, look up patients, open records, lab results and medications every one to three seconds (and make a mistyped login now and then). That is continuous, realistic traffic on the real workloads.
- The four agents poll every workload once a second and count what happens: **requests per second, outbound connections, data sent, failed logins**. The Live Monitoring page draws a small chart for each of these for each service, labelled LIVE_CLUSTER, with the age of the data. Nothing simulated can appear there.
- **The agents watch behaviour, never records.** The measurements have no field that can hold a patient record, and the agents' Kubernetes permissions cannot read the application's data. The dataset file is also fingerprinted, so a changed file is detected like any tampered program file.



SANDBOX-CLUSTER screenshot (the author's test cluster, not the presenter's): the Live Monitoring charts, one row per service, four charts each, telemetry age 0 s. The clinicians' traffic is visible as requests per second and data sent; outbound connections and failed logins stay flat.

Why not real patient data? Privacy law and ethics approval would be needed, and a platform that demonstrates attacks and isolation must never put a real person's record at risk. Synthea keeps the structure (clinical codes, lab values, medication lists) without a real person. Limits: it is a model, the sample is small and static, and it is not for clinical conclusions (docs/DATASET.md).

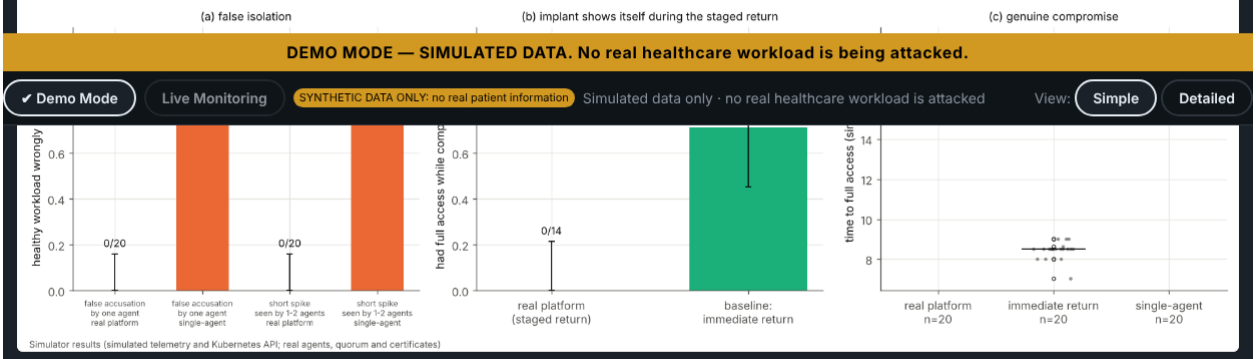
6. Results measured in the simulator

These are SIMULATOR results. The telemetry and the Kubernetes API are simulated; the agents, signatures, 3-of-4 agreement, certificates and the recovery pipeline are the real code. Times such as '1.2 s' say nothing about real Kubernetes, where a pod takes tens of seconds to start. What carries over is *who* gets isolated, who does not, and what survives a failure. 20 trials per case unless stated; everything is reproducible with `python3 scripts/evaluate.py` (docs/RESULTS.md).

Question	What the simulator showed
Does the full cycle work?	80 of 80 scenario trials went through detection, isolation, recovery and return; a genuine attack was isolated in about 1.2 s and back at full access in about 18.5 s of simulator time.
One lying agent?	never isolated a healthy workload (0 of 20); the lying agent was marked suspect after about 22 s and ignored after about 25 s.
A lying or crashed central controller (the alternative design)?	a lying controller isolated a healthy workload 20 of 20 times; with the controller crashed a real attack was not contained (0 of 20); with one agent crashed it was (20 of 20).
Cost of the safety?	containment takes about 1.5 s instead of 0.5 s.
Where does it break?	3 of 4 compromised agents isolate a healthy workload (20 of 20); 2 compromised agents plus one honest agent seeing a weak blip also do (20 of 20); 2 silent agents mean nothing is isolated (0 of 20). One compromised agent never broke anything.
A compromised agent that refuses to vote ('obstruct')?	the other three contained and recovered the workload in 20 of 20 trials with exactly 3 signers; its vote was never counted (0 of 20). It was **not penalised**: the platform has no rule for a missing vote.
False alarms?	in a mixed run no genuine attack was missed (0 of 30) but 50% of benign traffic spikes were isolated, all of them seen by 3 or more agents: a threshold detector cannot tell a unanimous harmless spike from an attack.
Optional 6-second re-check (off by default)?	false isolations 12 of 20 down to 9 of 20: every short blip filtered (3 of 11 down to 0 of 11), none of the 9 long bursts. It delays a network-only attack from about 1.0 s to 7.0 s, and an attacker sending 3 s bursts every 5 s is **never** contained with it on (0 of 20, against 20 of 20 with it off).
Targeted investigation?	did **not** reduce false isolations in my mix (4 of 20 both ways); it made slow-burn attacks isolate about 4.2 s sooner and turned silent ambiguity into a human-review flag.

4.11 What the safeguards buy: two evaluation-only baselines (Phase I) SIMULATOR

Evaluation-only baselines against the real platform (simulator, identical seeds)



Metric	Start event	End event	Denominator
Contained	attack injected	any agent isolates the workload within 40 s	all trials of the row
Time to isolate	attack injected	first agent isolates it (same as TTI above)	trials that isolated
Time to full access	attack injected	first agent confirms normal access (TTF)	trials that reached it (the table says n)
Full access while compromised	the dormant implant starts (injected after the stated delay, once the replacement began its return)	any 0.25 s sample in which the workload has **no isolation policy** (full access) **and** is compromised	trials in which the replacement reached the return stage (the "returning" trials; the table gives the number)
Seconds at full access while compromised	same start	sum of those samples x 0.25 s	same trials
Contained again / ended clean	the implant starts	a second isolation (epoch 2) / back at full access with a clean replacement	same trials
Healthy workload wrongly isolated	accusation signed / spike started	any agent isolates a workload that was never attacked within 25 s / 30 s	all trials of the row

Policy	Trials (denominator)	Contained	Time to isolate, simulator s	Time to full access, simulator s	Reached full access	Another workload isolated
real platform (3 of 4, staged return)	20	20/20 = 100% (95% CI 84-100%)	median 1.1 (IQR 1.1-1.1, min 1.1, max 1.5, n=20)	median 18.3 (IQR 18.0-18.5, min 17.5, max 19.0, n=20)	20/20 = 100% (95% CI 84-100%)	0/20
baseline 1: immediate reintegration (no stage gates)	20	20/20 = 100% (95% CI 84-100%)	median 1.1 (IQR 1.1-1.1, min 0.6, max 1.6, n=20)	median 8.5 (IQR 8.5-8.5, min 7.0, max 9.0, n=20)	20/20 = 100% (95% CI 84-100%)	0/20
baseline 2: single-agent decision (quorum 1)	20	20/20 = 100% (95% CI 84-100%)	median 0.5 (IQR 0.5-0.5, min 0.0, max 0.5, n=20)	median 15.5 (IQR 15.5-15.5, min 15.0, max 15.5, n=20)	20/20 = 100% (95% CI 84-100%)	0/20

Implant shows itself	Policy	Trials that reached the return stage (denominator)	Full access while compromised	Seconds at full access while compromised	Contained again	Ended clean
during the staged return (2-12 s after the return began)	real platform (3 of 4, staged return)	14	0/14 = 0% (95% CI 0-22%)	median 0.00 (IQR 0.00-0.00, min 0.00, max 0.00, n=14)	14/14 = 100% (95% CI 78-100%)	14/14 = 100% (95% CI 78-100%)
during the staged return (2-12 s after the return began)	baseline 1: immediate reintegration (no stage gates)	14	10/14 = 71% (95% CI 45-88%)	median 1.00 (IQR 0.19-1.00, min 0.00, max 1.50, n=14)	14/14 = 100% (95% CI 78-100%)	14/14 = 100% (95% CI 78-100%)
after the staged return would have finished (> 12 s)	real platform (3 of 4, staged return)	6	5/6 = 83% (95% CI 44-97%)	median 1.00 (IQR 0.81-1.00, min 0.00, max 1.00, n=6)	6/6 = 100% (95% CI 61-100%)	6/6 = 100% (95% CI 61-100%)
after the staged return would have finished (> 12 s)	baseline 1: immediate reintegration (no stage gates)	6	6/6 = 100% (95% CI 61-100%)	median 1.00 (IQR 0.81-1.19, min 0.75, max 1.50, n=6)	6/6 = 100% (95% CI 61-100%)	6/6 = 100% (95% CI 61-100%)

Situation	Policy	Trials (denominator)	Healthy workload wrongly isolated	Time to the wrong isolation, simulator s
one compromised agent signs a false accusation	real platform (3 of 4, staged return)	20	0/20 = 0% (95% CI 0-16%)	n/a (no trial reached it)
one compromised agent signs a false accusation	baseline 2: single-agent decision (quorum 1)	20	20/20 = 100% (95% CI 84-100%)	median 0.8 (IQR 0.8-0.8, min 0.2, max 0.8, n=20)
a short traffic spike seen by 1-2 agents	real platform (3 of 4, staged return)	20	0/20 = 0% (95% CI 0-16%)	n/a (no trial reached it)
a short traffic spike seen by 1-2 agents	baseline 2: single-agent decision (quorum 1)	20	20/20 = 100% (95% CI 84-100%)	median 0.8 (IQR 0.6-0.8, min 0.2, max 0.8, n=20)

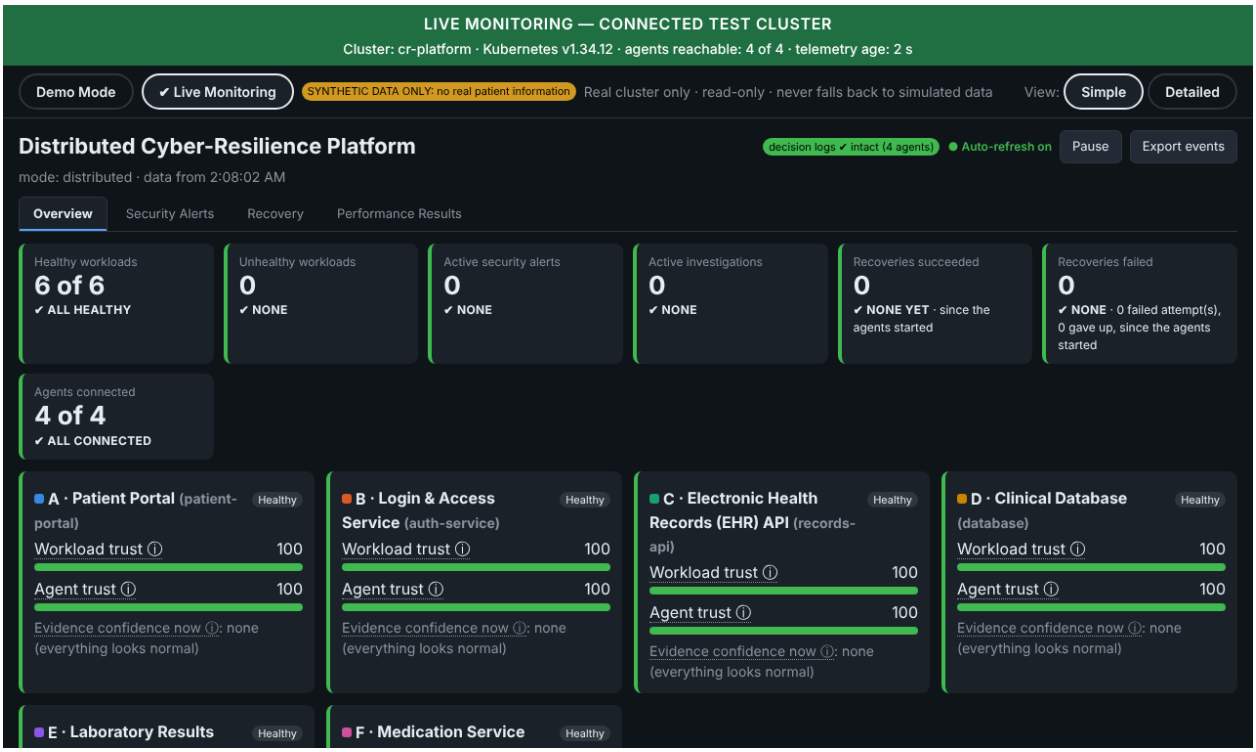
▶ **Explanation: how to read it and what the data shows**

Baselines (simulator): the same attacks run against designs with the gates removed (section 4.11 of the results), so the value of the 3-of-4 agreement and of the Recovery Check is measured, not assumed.

7. Results from a test cluster in the author's sandbox (SANDBOX-CLUSTER)

SANDBOX-CLUSTER results. These come from a real 4-node Kubernetes cluster (Minikube, Calico network policies) that the author built in a cloud sandbox on 2026-10-16. They are **not** simulator results and they are **not** the presenter's own live-verified results (section 8 stays empty). The sandbox differs from a laptop in ways listed in docs/SANDBOX-CLUSTER.md section 2 (for example Kubernetes 1.34.12 and no Metrics API).

Check	Result on the sandbox cluster
Deployment	six workloads, four agents, dashboard and two admission-webhook replicas all running, using the project's own scripts
Known-good file check (verify-baseline)	all six workloads match the manifest (five files each, including the dataset), before and after every incident
Agent permissions (verify-rbac)	27 of 27 checks pass for each of the six workloads
Admission webhook (verify-webhook)	7 of 7 steps pass, and 8 of 8 with the webhook switched off: an agent's isolation request is still refused when it is not backed by a 3-of-4 certificate
Network isolation (verify-isolation)	16 of 16 checks pass on two workloads with real Calico enforcement: user traffic blocked, agents still reach the workload
Controlled tripwire test	the real agents detected a harmless one-line change in the Laboratory Results Service and in the Medication Service, agreed (4 of 4), isolated, replaced and checked the workload, and restored access in four steps: about 82 seconds from the change to normal access, almost all of it Kubernetes' 30-second pod shutdown wait and the four 10-second stages
The Synthea data and clinician traffic	an 11-minute observation: every workload stayed healthy, nothing was isolated, at most 1 outbound connection (the limit is 8), at most 39 KiB/s sent (the limit is 256), no failed-login alarm
Bugs found and fixed	four deployment bugs were found only because it ran on a real cluster (two script bugs in the verification scripts, one in the upgrade path, one in the webhook order); each was fixed and has a test



SANDBOX-CLUSTER screenshot: Live Monitoring connected to the author's test cluster: green banner, cluster name and Kubernetes version, 4 of 4 agents reachable, six workload cards.

What this does not prove: behaviour on Windows/WSL and Docker Desktop, the presenter's Minikube version, kubectl top figures, and anything about a production hospital cluster (docs/SANDBOX-CLUSTER.md section 6).

8. Results verified on the real cluster

LIVE-VERIFIED results: none yet. This table is a placeholder for the presenter to fill with their own runs (docs/LIVE_DEMO.md, section 9). Nothing in it has been verified. Do not describe a simulator result as a live result.

Test (runbook step)	Date	Result	Evidence (screenshot or log)
Pre-flight: verify-baseline, verify-isolation	not run		
Healthy state, 4 of 4 agents	not run		
Agent goes offline, 3 of 4 carry on, agent rejoins	not run		
Tripwire test: real detection, isolation, recovery, return	not run		
Compromised agent (false accusation)	not run		
Forgery rejected	not run		
Obstruct plus tripwire	not run		
Centralized comparison	not run		

9. A suggested demonstration (about 10 minutes)

In Demo Mode (safe, repeatable):

Scenario in the picker	What to say
Patient records being copied out of the EHR API	three of four agents agree, isolate it, a clean copy is checked, access returns in steps
A security agent falsely accuses the Login & Access Service	one liar cannot reach three votes; the liar is marked suspect
A security agent goes offline, then the EHR API is attacked	three agents are still enough
A compromised agent refuses to vote while the EHR API is attacked	the other three still act; the dashboard shows the refusing agent as 'Disagrees' with its reason

On the real cluster (only if you have run and verified it first): the healthy state, deleting one agent pod, and the **tripwire test**: a single harmless line appended to a designated, unused marker file in one workload, which the real agents detect and handle with the real pipeline. Exact commands, what to point at, and how to reset are in `docs/LIVE_DEMO.md`.

The one exception to 'attacks only in the simulator'. The tripwire is not an attack: it is a harmless, operator-initiated change to one file that nothing reads, made with `python -c` (not a shell), refused for anything but the six healthcare workloads and for any other cluster. It triggers real isolation of a synthetic-data workload by design. It is documented as a controlled validation event with restore steps.

What the project deliberately does **not** contain: attack code, real patient data, or any way for the simulator's controls to reach a real cluster (the server refuses it, and tests prove it).

10. Limitations, stated plainly

- **Verified only on one test cluster.** Real pod start times, Calico enforcement and the admission webhook were exercised on the author's sandbox cluster (section 7), not on the presenter's laptop, not on Windows/WSL and not on a production hospital system.
- **Simulated telemetry in the measured results.** The numbers in section 6 come from invented signals; real traffic will differ, and the false-alarm rates depend on my choice of benign mix. On the sandbox cluster, healthy Synthea-based traffic raised no alarm.
- **Telemetry is reported by the workload itself.** A fully compromised workload could lie about itself, and all four agents read the same source.
- **Three or more compromised agents** can isolate a healthy workload; two silent or obstructing agents stop containment. The design tolerates one bad agent, not two.
- **A sustained harmless spike seen by everyone looks like an attack** to a threshold detector; the optional re-check helps with short spikes and can be evaded by short bursts.
- **A missing vote costs the agent nothing** (no trust rule for it).
- **Agents keep history in temporary storage:** a deleted agent pod rejoins with empty history.
- **Six workloads only**, watched by four agents. Not a compliance claim: the HIPAA Security Rule categories in docs/HEALTHCARE.md are a mapping for discussion, with gaps listed.

11. Questions the faculty may ask, with honest answers

Q. Why four agents and three of four?

Four agents can tolerate one that is wrong or down and still reach three. One bad agent can neither cause an isolation (it cannot get three votes) nor block one (the other three suffice). It is a deliberate, small choice; docs/RESULTS.md shows exactly where it stops working.

Q. What if two agents are compromised?

Then it breaks, and the results say so: two compromised agents plus one honest agent that sees a weak blip can isolate a healthy workload, and two silent agents mean nothing is isolated. The claim is tolerance of one failure, not two.

Q. Has this been tested on a real cluster?

On one test cluster, yes: I built a real 4-node Kubernetes cluster in a cloud sandbox and ran the deployment, the verification scripts, the harmless tripwire test and an 11-minute observation with realistic Synthea traffic (section 7, labelled SANDBOX-CLUSTER). The measured numbers in section 6 are simulator results (real agents and cryptography, simulated telemetry and Kubernetes). I have not yet verified anything on my own laptop cluster, so the live-verified table in section 8 is empty until I run it.

Q. Why not one smart central controller?

It is a single point of failure and of compromise. In my simulation a lying controller isolated a healthy service 20 of 20 times and a crashed one never contained a real attack; the distributed design did neither. The price is slower containment (about 1.5 s against 0.5 s in the simulator).

Q. How does it recognise an attack?

Four signals measured by every agent independently: unexpected connections or traffic, unexpected programs, files that differ from the known-good copy, and bursts of failed logins. Weak or disputed evidence triggers a short investigation first.

Q. What about false alarms?

They exist. In my stress mix 50% of harmless spikes that three or more agents saw were isolated. An optional 6-second re-check lowers that (12 of 20 to 9 of 20) but delays some real attacks by about 6 s and can be evaded by short bursts, so it is off by default.

Q. What does isolating a workload do to patients and staff?

It cuts the workload off the network, so services that depend on it stop working: for example isolating the EHR API means the Patient Portal cannot show records. The dashboard shows this impact in advance. It is deliberate: a compromised system must not keep running.

Q. How do you know a recovered workload is clean?

A Recovery Check: healthy, files match the known-good copy, behaves normally, and it is a new instance. If it fails it is retried a limited number of times, then marked 'needs human attention' and stays isolated; it is never shown as healthy.

Q. Can a compromised agent forge another agent's messages?

Not undetected: messages are signed and agents use mutual TLS, so forged ones are rejected and the sender loses trust. A compromised agent can still lie with its own genuine identity; that is what the 3-of-4 rule handles.

Q. Is this HIPAA compliant?

No claim is made. docs/HEALTHCARE.md maps what the platform does to HIPAA Security Rule categories (access control, audit, integrity, contingency planning, transmission security) and lists the gaps; compliance needs much more than this project.

Q. Is any patient data real?

No. Every patient comes from Synthea, an open-source generator of invented patients, and is labelled synthetic on every page and in every service answer. The security agents watch behaviour (connections, programs, files, logins) and never read patient records.

Q. Is the 'tripwire' an attack?

No. It is one harmless line appended to an unused marker file, run by the operator on purpose, so the real agents have something genuine to detect. It is a narrow, documented exception, with restore steps.

Q. What would you do next?

Measure on my own laptop cluster and a larger one, tune thresholds on real traffic, add more workloads, add stronger sensors, and evaluate with realistic (still synthetic) workloads. These are future work, not done.

12. Glossary

Term	Plain meaning
Workload	one application service running in the cloud (portal, login, records API, database)
Agent	a small security program next to a workload; four identical peers
Evidence	a signed report from one agent about what it sees
Agreement Reached (3 of 4)	at least three agents signed a vote; also called a quorum
Isolation	cutting a workload off the network (reversible)
Recovery Check	tests that a replacement is healthy, matches the known-good copy and behaves normally
Normal Access Restored	the last step of the staged return, after limited, monitored and peer-confirmed access
Simulator	the test world with simulated telemetry and Kubernetes, used for all measured results
Synthetic data	made-up patient records (from the Synthea generator); no real person is represented
SANDBOX-CLUSTER	results from the author's own test Kubernetes cluster in a cloud sandbox; not simulator results, not the presenter's live results

Where to find things: `docs/RESULTS.md` (all numbers and graphs), `docs/HEALTHCARE.md` (requirements and safeguards), `docs/DASHBOARD.md`, `docs/LIVE_DEMO.md` (runbook), `docs/PROGRESS.md` (what was done, tests, limits).